

Decorators aur call / apply

Decorator kya hai?

Aisa function jo **doosre function ko wrap** karke uska behavior badal deta hai, bina uska code chhede.

Example: Caching decorator

Agar koi function bhaari hai aur same input par same result deta hai, to result yaad rakh lo.

javascript

```
function cachingDecorator(func) {  
  let cache = new Map();  
  return function (x) {  
    if (cache.has(x)) return cache.get(x);  
    let result = func(x);  
    cache.set(x, result);  
    return result;  
  };  
}  
  
slow = cachingDecorator(slow);
```

Fayde:

- Kisi bhi function par reuse kar sakte ho.
- Original function simple rehta hai.
- Kai decorators combine kar sakte ho.

Problem: object method me `this` kho jaata hai

Wrapper ne `func(x)` bola to `this` `undefined` ho jaata hai. Object ke method me `this.something` toot jaata hai.

Solution: `func.call`

`this` ko manually set karke function chalane ka tareeka.

javascript

```
func.call(context, arg1, arg2);
```

Ye `func(arg1, arg2)` jaisa hi hai, bas `this = context` set ho jaata hai.

javascript

```
function say(phrase) {  
  console.log(this.name + ": " + phrase);  
}  
say.call({ name: "John" }, "Hello"); // John: Hello
```

Wrapper me use:

javascript

```
let result = func.call(this, x);
```

Multiple arguments

`Map` ka key ek hi value hota hai, to arguments ko jod kar ek key banao (jaise `"3,5"`). Saare arguments pass karne ke liye `func.call(this, ...arguments)`.

`func.apply`

javascript

```
func.apply(context, args); // args array ya array-jaisa
```

`call` aur `apply` me sirf ye farak hai:

- `call` -> arguments **alag-alag** lete hain.
- `apply` -> arguments **ek array** me lete hain.

Call forwarding (sab kuch aage bhej dena):

javascript

```
let wrapper = function () {  
  return func.apply(this, arguments);  
};
```

Bahar se dekhne par wrapper aur original function me koi farak nahi dikhta.

Method borrowing

`arguments` asli array nahi hota, isliye `arguments.join()` nahi chalta. Array ka method udhaar le lo:

javascript

```
[].join.call(arguments);
```

Ye isliye chalta hai kyunki `join` andar se sirf `this[0]`, `this[1]`... use karta hai.

Dhyan rakhne wali baat

Decorator wrapper hota hai. Agar original function par koi property thi (jaise `func.count`), to wrapper me wo nahi milegi.

Task wale important decorators

Spy: har call ke arguments (`wrapper.calls`) me save karta hai (testing me kaam aata hai).

Delay: function ko (`ms`) baad chalata hai.

javascript

```
function delay(f, ms) {  
  return function () {  
    setTimeout(() => f.apply(this, arguments), ms);  
  };  
}
```

Debounce: jab tak calls aati rahein, wait karo. (`ms`) tak shanti rahe tab **sirf aakhri call** chalao. (Search box me typing khatam hone par request bhejna.)

javascript

```
function debounce(func, ms) {  
  let timeout;  
  return function () {  
    clearTimeout(timeout);  
    timeout = setTimeout(() => func.apply(this, arguments), ms);  
  };  
}
```

Throttle: function ko (`ms`) me **maximum ek baar** chalao. Pehli call turant, beech ki calls ignore, aur aakhri call cooldown ke baad. (Mouse move par heavy update.)

Debounce = "ruko, shaant hone do, fir ek baar". Throttle = "chahe jitni calls aayein, itne time me bas ek".

Yaad rakho

- Decorator = function ko wrap karke naya behavior dena.
- `call` se `this` set karo, `apply` array le leta hai.

- Forwarding ke liye `apply(this, arguments)`.